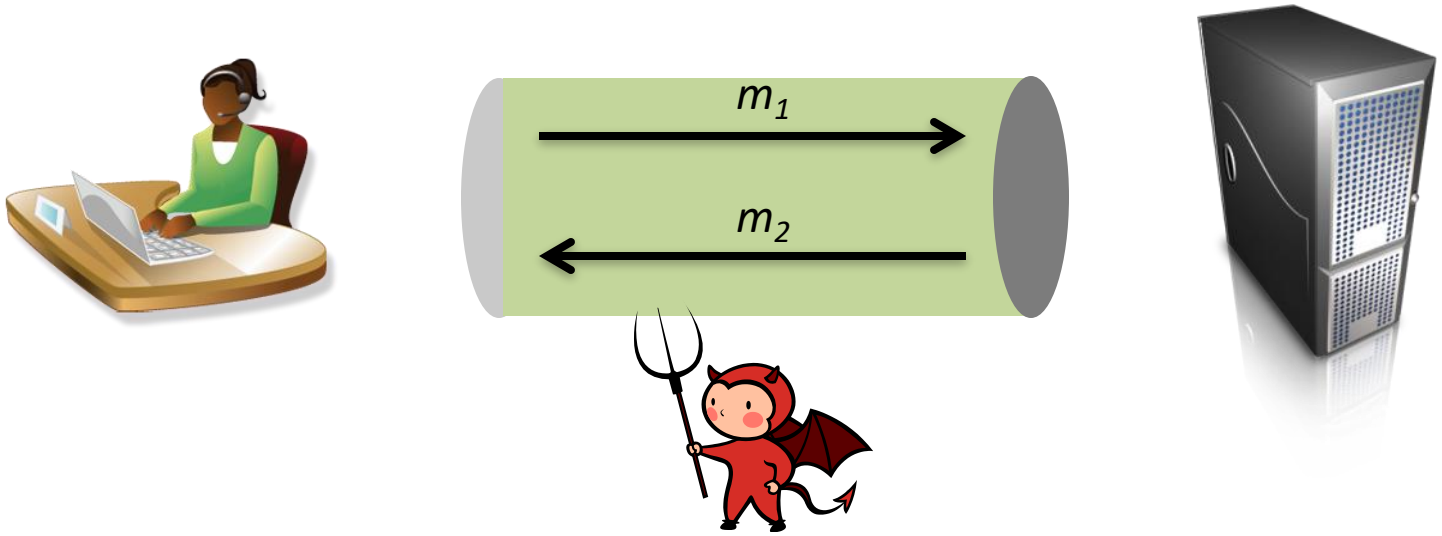
IY3660 – Applications of Cryptography

Week 4
Security models and attack complexity

Elizabeth Quaglia
Information Security Group

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Our main application: secure communications



Recall that in our analysis and selection of cryptographic schemes for setting up a secure communications channel, we need to careful consider:

- **What should the security goals be?**
- **What capabilities does the adversary have?**

Cryptographic security models

We must precisely define

- The **goals** of the adversary:
  - key recovery? decryption? partial information? message forgery?
- The adversary's **capabilities**:
  - ciphertext only? known plaintext? chosen plaintext? chosen ciphertext?

Then under a well-defined model, we try to **prove** that the scheme is **secure**:

- What we mean by secure also needs to be precisely defined
- Note: the notion of proof in cryptography differs from the one we are used to from mathematics!

Cryptographic security games

- These proofs will be carried out based on **security games**:
- There is an abstract entity called an **oracle***,* which implements the cryptographic functionality we want to analyse (and maybe some idealised functionality)
- The adversary can interact with the oracle, sending **queries***,* according to their defined **capabilities** (e.g., allowed to ask for encryption? for decryption?)
- **The game**: the oracle initialises, the adversary interacts with it making q queries, receiving the corresponding responses, and it then produces an output (maybe after some computation) – all per the rules of the game
- The adversary's **goal** is to win the game – this is quantified by the notion of **advantage** (indicating how much better they can do by playing the game, compared to random guesses)
- The cryptographic scheme is considered **secure** if this advantage is **negligible**

Oracles in cryptography

oracle: *(in ancient Greece) a place where people could go to ask the gods for advice or information about the future; the priest or priestess through whom the gods were thought to give their message*

In modern cryptography, an **oracle** is an abstract concept, implementing some cryptographic functionality, with which adversaries can interact with, in a **black box** manner:

- They send a query and get a truthful response (and nothing else)
  - E.g., here is a message; encrypt it with your symmetric encryption scheme (with a secret key I don't know), and send me back the ciphertext…
  - E.g., this is a message and a MAC tag; please confirm whether the MAC verifies correctly

- We will often have that a cryptographic implementation that behaves like an oracle
- This functionality and interface is then modelled as an oracle, for the purpose of security proofs
- Even if you think that a particular oracle functionality is unrealistic, if you can prove security under some (at least currently) unrealistic and powerful adversarial assumption, great!

Security notions

<GOAL>-<CAPABILITY> security

<GOAL>:

- OW: message recovery
- IND: indistinguishability
- etc

<CAPABILITY>:

- PASS (or COA): passive, ciphertext-only
- KPA: known-plaintext attack
- CPA: chosen-plaintext attack
- CCA: chosen-ciphertext attack
- etc

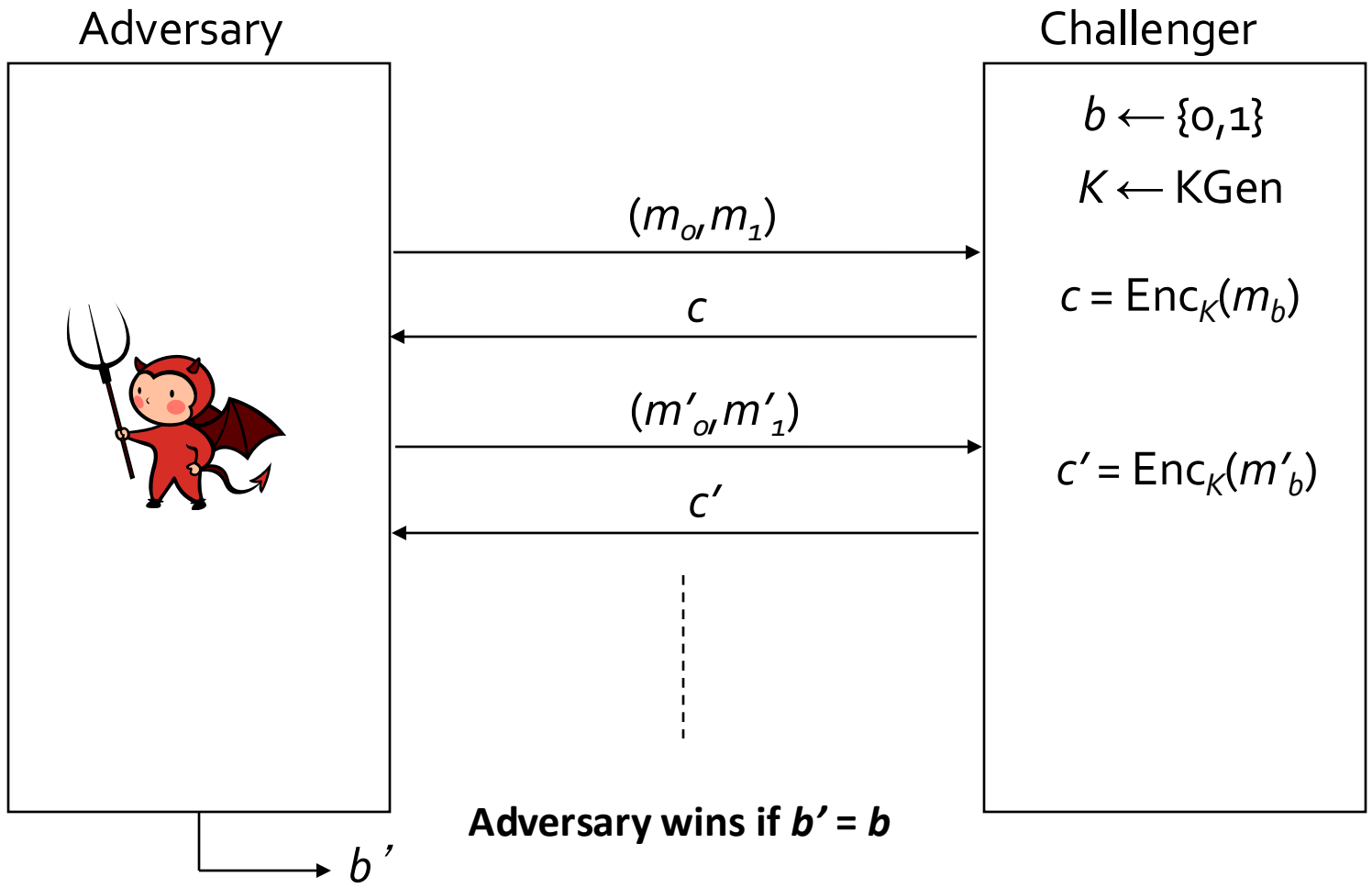
Security notions and examples

<GOAL>-<CAPABILITY> security

- **OW-PASS secure**: scheme is secure against passive adversary (i.e., no access to the oracle) attempting to recover message
- **IND-CPA secure**: scheme is semantically secure; adversary with access to encryption oracle cannot distinguish between the encryption of two messages

What security notion does the One-time pad satisfy?
We will see more in the course

Example: indistinguishability (IND) under chosen-plaintext attack (CPA) security game for encryption schemes



Initialise: the oracle (challenger) draws at random a secret encryption key K and a bit b= 0 or 1

If this Advantage is small, then we say that Enc is **IND-CPA secure**

$$\mathrm{Adv}_E^{IND\text{-}CPA}(A) := 2.\Pr[b'=b] - 1$$

Attack complexity

When considering the security of cryptographic primitives or schemes, we are interested in measuring the resources required by the attacker

For example, in the IND-CPA game:

- how many pairs (m_0, m_1) does Adversary send to the oracle?
- after collecting all the responses, what kind of computations are required to produce its guess $\boldsymbol{b}'$?

Attack complexity

Time complexity:

- Measures the computational complexity of the attack
  - e.g., in an exhaustive key search, the time complexity is on the order of 2^k encryption operations (where k is the length of the key in bits)
- Could be measured in computation cycles required
  - We need then to estimate equivalent number of encryption operations
    - If less than 2^k, then the cipher may be considered broken

Attack complexity

Data complexity:

- How much data is required to run the attack
- Typically corresponds to the number of queries to the oracle (in chosen-plaintext / chosen-ciphertext attacks)
- It could also be simply the amount of data observed by the attacker (in known-plaintext / ciphertext only attacks)

Memory (or space) complexity:

- How much memory is required to run the attack

Attack complexity

- Most times, when we state the “complexity” of an attack, we are often referring to time complexity
- Data and memory complexity are also relevant
- All must be considered to assess whether the cipher may be broken!

Exhaustive key search

- Attack strategy:
  - Given a plaintext-ciphertext pair (p, c), such that $c = E_K(p)$, where K is a n-bit secret key, an exhaustive key search tries all possible keys until one is found that matches
- Attack complexity:
  - **Data**: one plaintext-ciphertext pair (in some cases, another 1-2 extra required)
  - **Memory**: nil (well, you need to store (p, c), but memory requirement is negligible)
  - **Time**: on the order of 2^n encryption operations

Precomputation-based key-recovery

- Attack strategy:
  - Pick a plaintext p
  - For all possible k, precompute $c = E_k(p)$, and store in a table all pairs (c, k)
  - Then in a chosen-plaintext attack, ask the oracle to encrypt your chosen p with the secret key K (as $c = E_K(p)$)
  - Search for that c in your table and return the corresponding k as the secret key K

- Attack complexity:
  - **Data**: again, one plaintext-ciphertext pair (but maybe 1-2 extra queries to confirm among candidates)
  - **Memory**: you need to store 2^n pairs (c, k) (note: if $n = 128$, this is a **very large** table)
  - **Time**: a simple table look-up for c
    (note: we do not consider the time required to **build** the table as contributing to the time complexity of the attack...)

Recap

- In modern cryptography, we claim security wrt a **goal** and the adversary's **capability**
- Security definitions are formal and are typically described as games
- Attacks can require time, data and memory